NAME: _____ **BU ID:**_____

Final Exam – Tuesday, December 14th, 2021 – 3pm-5pm
EC327 Introduction to Software Engineering – Fall 2021

Total: 150 points

****MAKE SURE TO READ THE FOLLOWING BEFORE STARTING YOUR EXAM****

Exam Rules:

- **Save your work often during the exam**. You would not want to lose any work. Pay attention to where you are saving it as well!!

You **CAN** use the following during the exam:
- Linux programs (gcc, vi, gedit, gdb, emacs, etc.) on PHO305/307 machines,
- The Forouzan course textbook (1st edition)
- **Unlimited** PRINT material that you bring to the exam yourself (you cannot share material)

- You **CANNOT** use hardware calculators, cell phones, laptops, any other electronic devices.

- You have until **5:00pm (SHARP)** to finish and submit your exam. Please follow the submission instructions below. Your submission will have a timestamp, so make sure to submit before the deadline. We reserve the right to NOT accept submission after the deadline.

- Do not spend too much time on any one question, you can always return to it later. **Focus on getting code that compiles and is well documented in the event that you cannot get the correct functionality. CODE MUST COMPILE OR ZERO CREDIT WILL BE GIVEN.**

- **Question 0** is an example so that you see how to submit. **IT IS AN EXAMPLE NOT A QUESTION.**

- Make sure to put all the code and all the files you need to submit in your exam directory, e.g., in *jbond_0007_002_final.* (See *Question 0* for more details about submission.)

- You are not allowed to access any other directories on the engineering grid (such as /ad/eng folders) or ssh to any machines. All actions on the lab machines are being logged during the exam.

GOOD LUCK!

Q0. Submission Procedure and Finding Your Way Around Linux.

*****This Question is An Example of How To Submit Your Work *****

a. Open a terminal window.

b. Create a folder named **<yourBUusername>_<last4digitsofBUID>_<machineNumber>_midterm**. For example, student *James Bond* with BU username *jbond* , BU ID U*12340007,* and machine number *002* (type **hostname** at Unix prompt to find computer name) should type:

```
mkdir jbond_0007_002_final
```

*** Make sure you put in your own name and number here, and not those of James Bond's!!!*

c. Change directory to your final directory:

```
cd jbond_0007_002_final
```

d. Create `test.cpp` file in your directory, and write the following code:

```
#include <iostream>
using namespace std;
int main()
{
  cout<< "TEST EC327 FINAL" << endl;
  return 0;
}
```

e. Zip your directory and submit via the following steps:

```
cd ..
ls
```

*** You should see your jbond_0007_final directory listed now.*

```
zip -r jbond_0007_002_final.zip ./jbond_0007_002_final/*
```

*** Make sure to replace James Bond's folder name with your folder name.*

f. Check that the zip file exists and contains all of the files:

```
ls
```

*** You should see jbond_0007_final.zip*

```
less jbond_0007_002_final.zip
```

*** This will list all the files in the zip. Make sure it contains all your files. ('q' to quit "less" program)*

g. Submit your solution:

Submit your solution by uploading it to the **Final Exam Portal on Blackboard**. On the EC327 page, go to "Content >> Exams >> Final >> Final Exam Submission." You will simply click "Browse My Computer" to select your zip file and then click "Open". Finally, click the green "Submit" to complete your submission.

You are allowed to submit your solutions as many times as you want; however, only the last submission will be graded. For each subsequent submission, please include a revision number in the filename so that we will know which submission is the latest (for example, jbond_0007_002_final_rev3.zip, etc.).

The upload system will prevent you from submitting a file with the same name as a file that has been already submitted. (This prevents you from overwriting previous submissions.) Hence the need to include revision numbers as discussed above. Make sure the file size as reported by the submission system matches your expectations!

Q1. Encapsulation [25 points] (15 minutes)

So no one told you life was going to be this way.
Your job's a joke, you're broke, your love life's DOA.
It's like you're always stuck in second gear,
Well, it hasn't been your day, your week, your month, or even your year.

But, I'll be there for you, when the rain starts to pour.
I'll be there for you, like I've been there before.
I'll be there for you, cause you're there for me too.

You are given a main program in `Q1.cpp`. In this file there is a class called *Superhero* and a *main* function. It should compile and run. Make sure that it does compile BEFORE proceeding.

You now need to create a new class called *Ironman*. The code illustrates where this class should go.

Superhero has three member variables: *iq*, *strength*, and *numPowers*. They are classified as private, protected, and public respectively. YOU CANNOT CHANGE THIS DESIGNATION.

You need to do the following:

1. You need to add the *Ironman* class with only ONE additional private variable: string id. Initialize the other variables as detailed in the comments.
2. You need to write the NON-MEMBER functions iq_getter, iq_setter, and id_getter functions where designated in the comments.
3. Modify *Superhero* and *Ironman* to work with the code in main WITHOUT changing main or the access level of the variables in *Superhero* or *Ironman*. For example you need to add a getter and setter method for *strength*.

When you are done, uncomment the lines in main. **Fix the typos** in the commented out section. DO NOT HOWEVER add any more lines to main. Just fix the typos. DO NOT add any other cout statements.

You should submit the following files for this problem:

`Q1.cpp`

Q2. Inheritance Basics [20 points] (10 minutes)

Figure 1: Nintendo Virtual Boy - 1995

You are given a *Videogame* class and *main* function. This is in the file `Q2.cpp`. It should compile and run. Make sure that it does compile BEFORE proceeding.

You CANNOT change the access level of the variables in *Videogame*.

You now need to add a *DonkeyKong* class where indicated. This class should be derived from *Videogame*. It needs to have the following characteristics:

1. It needs to be able to set the *cheatcode* in the constructor (without changing its access level)
2. It needs a string get_character() function that returns “Jumpman”.
3. It needs setter and getter methods for *highscore*.

Uncomment the code in main (fix typos). You also need to write some of the main code yourself according to the instruction in the comments.

Does the output make sense? You may need to modify the Videogame class at this point if functions are not returning what you expect. Maximum points are awarded for minimal changes.

You should submit the following files for this problem:

`Q2.cpp`

Q3. Exceptions, Destructors, and Casting [30 points] (20 minutes)

You are given a *Student* class and *main* function. This is in the file `Q3.cpp`. It should compile and run. Make sure that it does compile BEFORE proceeding.

FOLLOW THESE STEPS. You need to do the following:

1. Add a static int member variable called *objects* to the student class. Initialize it to zero. Think about where it needs to be initialized.
2. Increment *objects* every time a new *Student* is created.
3. Add a destructor to *Student*. Make sure to decrement the number of *objects* before the cout statement provided.
4. Create an *ENGMajor* object. It should be derived from *Student*. It should just have a destructor and override the *changePartyCount()* method. The destructor should just have the provided cout statement and the *changePartyCount()* method should throw a *PartyException* (ENGMajors can't party).
5. Create a *PartyException*. It should be derived from exception. The body of the exception is provided.
6. Fill in the body of the *saturdayNightPlans* function as described.
7. Uncomment main. There are NO typos in main.
8. Make sure the program behaves as expected. You may need to make minor changes to code outside of main to get this to happen.

You should submit the following files for this problem:

`Q3.cpp`

Q4. Operator Overloading and Dynamic Casting [20 points] (15 minutes)

In this problem you will overload the following operators**: +, *, and /** for combining two *Students*. *Student* is an abstract base class and hence you have to use one of the two derived classes, *Slacker* and *MedStudent*.

This problem requires you to:

1. Create the following **non-member** overloaded operator functions:
    a. Student *operator+(Student &firstStudent, Student &secondStudent)
        i. This function should return a new *Slacker* **if either** argument is a *Slacker*. Otherwise return a *MedStudent*.
    b. Student *operator/(Student &firstStudent, Student &secondStudent)
        i. Should return a new Object of the type of the denominator.
    c. Student *operator*(Student &firstStudent, Student &secondStudent)
        i. This function should return a *Slacker* only **if both** arguments are *Slackers*. *MedStudent* otherwise.
2. Fix the 6 lines in the main function to perform the operations listed. **DO NOT CHANGE ANY OTHER LINES IN MAIN.**

IF YOU CHANGE MAIN TO HARDCODE THE SOLUTIONS YOU WILL LOSE ALL POINTS FOR THIS QUESTION.

This material is all in the file `Q4.cpp`. It should compile and run. **Make sure that it does compile BEFORE proceeding.**

You should submit the following files for this problem:

`Q4.cpp`

Q5. Templates and Arrays/Pointers [20 points] (15 minutes)

This problem requires you to make two functions with the following prototypes:

1. T *destroy(T list[ ], int listSize, int &destroyedSize, T target)
2. T *replace(T list[ ], int listSize, T insert, T target)

The destroy function needs to remove the target element(s) from the original list. The replace function needs to replace the target element(s) with the insert element.

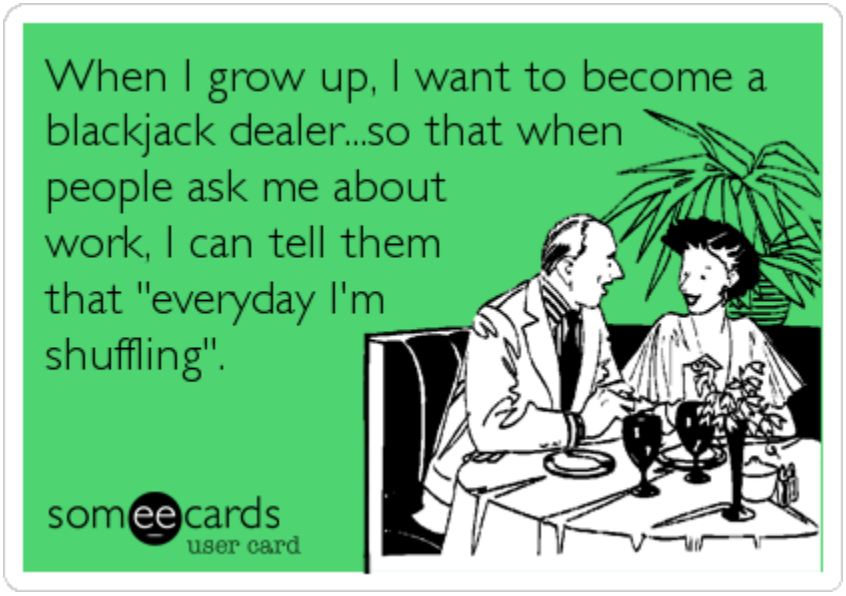
More information is provided in the code**. If you have questions about the functionality please ask the staff before you begin.**

Both functions need be correct for full credit. The algorithmic complexity does not matter (i.e. just get it right). You will see if it is correct when you uncomment the main function code.

You should submit the following files for this problem:

`Q5.cpp`

Q6. Data Structures/Coder Freedom [25 points] (20 minutes)



In this problem you will create a basic game of Blackjack[1]. However you input what card you receive (as opposed to it being randomly dealt to you).

2-10 are cards of that face value.

11, 12, 13 are the Jack, Queen, and King and are 10 points each.

14 is an Ace. If the total in your hand is less than or equal to 10 it will be worth 11. If the total is more than 10 it is one point. For example is the total is 2 and you get an Ace, the total is now 13. If the total was 13 and you get an Ace, the total is now 14. **NOTICE THAT ONCE AN ACE CONTRIBUTES TO THE SUM, IT DOES NOT CHANGE VALUE AGAIN.** For example 2, 14, 10 is a bust since you can't count the Ace as 1 point once you "draw" the 10. I busts since the sum would be 23 (2+11+10) in this case. <u>This is NOT like true Blackjack in this way.</u>

You need to do basic error checking to make sure that an integer entered is not outside the range of legitimate cards (you only need to check for integers). Also you need to check that 4 of that number have not already been used (there are only 4 of each type of card in the deck).

You have complete freedom on this problem. **BUT you must only produce the sample output shown in the file provided to you. See this sample for example scenarios.**

You should submit the following files for this problem:

`Q6.cpp`

---

1. Notice this is not true Blackjack due to how Aces are handled.

Q7. Multiple Choice [10 points] (15 minutes)

Open Q7.txt. It currently lists 10 responses (all "f", like "fail" which is what will happen if you don't change them since there are no 'f' answers). Modify this file ONLY by changing the response to the correct letter. DO NOT change the spacing or case of the letters.

1. What is the difference between a friend function and a member function?
   a. Member functions in a derived class have access to the private variables of the base class. Friends don't.
   b. All functions in base classes are friend functions for derived classes.
   c. Member functions have access to the classes' private variables while friend functions need not be member functions (can be defined outside of the class) to have access to the private variables.
   d. There is no difference.
   e. All friends are members but not all members are friends.

2. Assume that double precision floating point numbers are eight bytes each and the starting address of an array is 1002500 in a byte addressable memory. Assuming nPtr points to the beginning of the array, what address is in nPtr + 8?
   a. 1002508
   b. 1003012
   c. 1002564
   d. 0
   e. 1002516

3. _____ involves using a _____-class pointer or reference to invoke _____ functions on _____-class and _____-class objects.
   a. Inheritance, Derived, Private, Derived, Base
   b. Polymorphism, Base, Virtual, Base, Derived
   c. OOP, Simple, Virtual, Friend, Base
   d. Polymorphism, Derived, Virtual, Base, Derived
   e. Inheritance, Base, Virtual, Base, Derived

4. The _____, _____, and _____ of an operator cannot be changed by overloading the operator.
   a. Distribution, arrangement, locality
   b. Meaning, style, panache
   c. Semantics, syntax, grammar
   d. Precedence, associativity, "arity"
   e. Look, feel, style

5. Which of the following is NOT a common type of exception:
   a. Array out of bounds
   b. Division by zero
   c. Invalid input
   d. Invalid parameters
   e. Memory already in use

6. When a _____-class object is _____ the _____ are called in _____ order of the _____.
    a. Derived, destroyed, destructors, reverse, constructors
    b. New, created, constructors, reverse, destructors
    c. Derived, created, constructors, reverse, destructors
    d. New, destroyed, destructors, same, constructors
    e. Circle, created, GeometricObjects, reverse, Rectangles

7. Put these 6 phases in the correct order:
    a. Executing, Assembling, Preprocessing, Linking, Loading, Compiling
    b. Preprocessing, Assembling, Linking, Compiling, Loading, Executing
    c. Preprocessing, Compiling, Assembling, Linking, Loading, Executing
    d. Preprocessing, Assembling, Compiling, Loading, Executing, Linking
    e. None of the above

8. You used the following STL object in PA4
    a. Vector
    b. List
    c. Map
    d. Deque
    e. Set

9. In a class, member variables are often called its _____, and its member functions are sometimes referred to as _____.

    a. behavior, state
    b. none of these
    c. state, behavior
    d. data, activities
    e. values, methods

10. The following was NOT a role in the Class Project
    a. Project Lead
    b. GUI Designer
    c. Documentation Manager
    d. Class Designer
    e. Interface Designer

End-of-Exam Checklist

- Your midterm folder should contain (case-sensitive):

  - `Q1.cpp`
  - `Q2.cpp`
  - `Q3.cpp`
  - `Q4.cpp`
  - `Q5.cpp`
  - `Q6.cpp`
  - `Q7.txt`

Zip your exam folder and submit to the **Final Exam Portal on Blackboard**, following the procedure discussed in Question 0. (Remember that you'll need to use revision numbers on the filename if you've already submitted your exam.)

- **Before logging off, check with the staff to make sure that your exam has been received!**